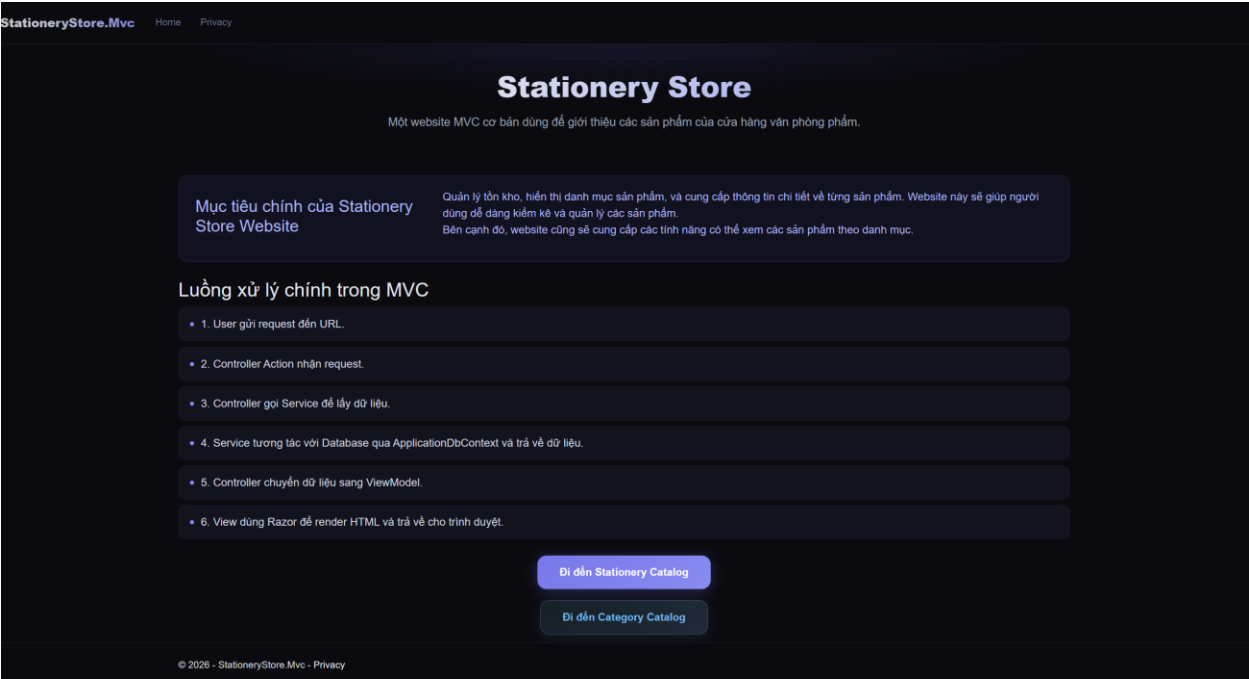


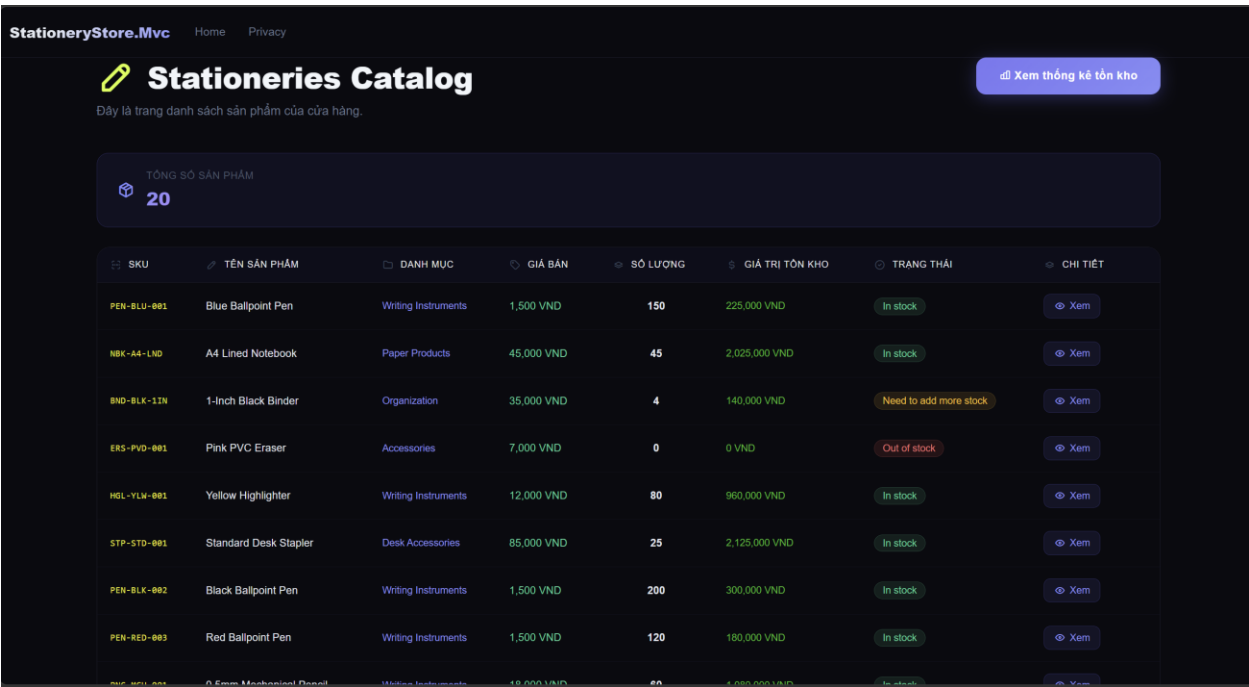
Link github: https://github.com/tin1508/Asp.Net_TTH/tree/main/Labs_2

Câu 1: Mini Stationery Store Catalog MVC

Trang /:



Trang /Stationeries:



Trang /Stationeries/Detail/id để xem chi tiết văn phòng phẩm.

Với trạng thái còn hàng (In Stock):

StationeryStore.Mvc

Home

Privacy

Stationery Detail

Thông tin chi tiết của sản phẩm trong kho.

←

Quay lại danh sách

Trạng thái tồn kho

In stock

No need to add more stock.

Giá trị tồn kho hiện tại

2,025,000 VND

Tính theo giá bán và số lượng tồn kho hiện có.

Cập nhật lần cuối

5/18/2026 10:28:11 AM

Thông tin sản phẩm được cập nhật gần nhất.

Với trạng thái cần nhập hàng thêm (Need to add more stock):

StationeryStore.Mvc

Home

Privacy

Stationery Detail

Thông tin chi tiết của sản phẩm trong kho.

←

Quay lại danh sách

Trạng thái tồn kho

Need to add more stock

Please add more stock. Current quantity is 4, minimum stock is 10.

Giá trị tồn kho hiện tại

140,000 VND

Tính theo giá bán và số lượng tồn kho hiện có.

Cập nhật lần cuối

5/18/2026 10:28:11 AM

Thông tin sản phẩm được cập nhật gần nhất.

Với trạng thái hết hàng (Out of stock):

StationeryStore.Mvc

HomePrivacy

Stationery Detail

Thông tin chi tiết của sản phẩm trong kho.

←

Quay lại danh sách

ID	4
SKU	ERS-PVD-001
Tên sản phẩm	Pink PVC Eraser
Danh mục	Accessories
Nhà cung cấp	EraserTech
Giá bán	7,000 VND
Số lượng tồn kho	0
Mức tồn tối thiểu	50
Giá trị tồn kho	0 VND
Trạng thái	Out of stock
Gợi ý nhập hàng	Please add more stock because the product is out of stock.
Cập nhật lần cuối	5/18/2026 10:28:11 AM

Trạng thái tồn kho

Out of stock

Please add more stock because the product is out of stock.

Giá trị tồn kho hiện tại

0 VND

Tính theo giá bán và số lượng tồn kho hiện có.

Cập nhật lần cuối

5/18/2026 10:28:11 AM

Thông tin sản phẩm được cập nhật gần nhất.

Trang /Stationeries/Stats:

StationeryStore.Mvc

HomePrivacy

Stationery Stats

Trang này giúp bạn xem nhanh tổng quan tình hình tồn kho của cửa hàng.

←

Quay lại danh sách

20

TỔNG SỐ LOẠI SẢN PHẨM

1789

TỔNG SỐ LƯỢNG TỒN KHO

58,585,000

VND

TỔNG GIÁ TRỊ TỒN KHO

1

SẢN PHẨM ĐÃ HẾT HÀNG

1

SẢN PHẨM CẦN NHẬP THÊM

Đánh giá tình hình tồn kho

Có sản phẩm đã hết hàng. Cần kiểm tra và nhập thêm sớm.

Tổng số loại sản phẩm	20
Tổng số lượng tồn kho	1789
Tổng giá trị tồn kho	58,585,000 VND
Số sản phẩm đã hết hàng	1
Số sản phẩm cần nhập thêm	1

Trang /Stationeries/StationeryJson:

```
localhost:5147/Stationeries/StationeryJson

Pretty-print ✓

[
  {
    "id": 1,
    "sku": "PEN-BLU-001",
    "name": "Blue Ballpoint Pen",
    "categoryId": 1,
    "price": 1500,
    "quantity": 150
  },
  {
    "id": 2,
    "sku": "NBK-A4-LND",
    "name": "A4 Lined Notebook",
    "categoryId": 2,
    "price": 45000,
    "quantity": 45
  },
  {
    "id": 3,
    "sku": "BND-BLK-1IN",
    "name": "1-Inch Black Binder",
    "categoryId": 3,
    "price": 35000,
    "quantity": 4
  },
  {
    "id": 4,
    "sku": "ERS-PVD-001",
    "name": "Pink PVC Eraser",
    "categoryId": 4,
    "price": 7000,
    "quantity": 0
  },
  {
    "id": 5,
    "sku": "HGL-YLW-001",
    "name": "Yellow Highlighter",
    "categoryId": 1,
    "price": 12000,
    "quantity": 80
  },
  {
    "id": 6,
    "sku": "STP-STD-001",
    "name": "Standard Desk Stapler",
    "categoryId": 5,
    "price": 85000,
    "quantity": 25
  },
  {
    "id": 7,
```

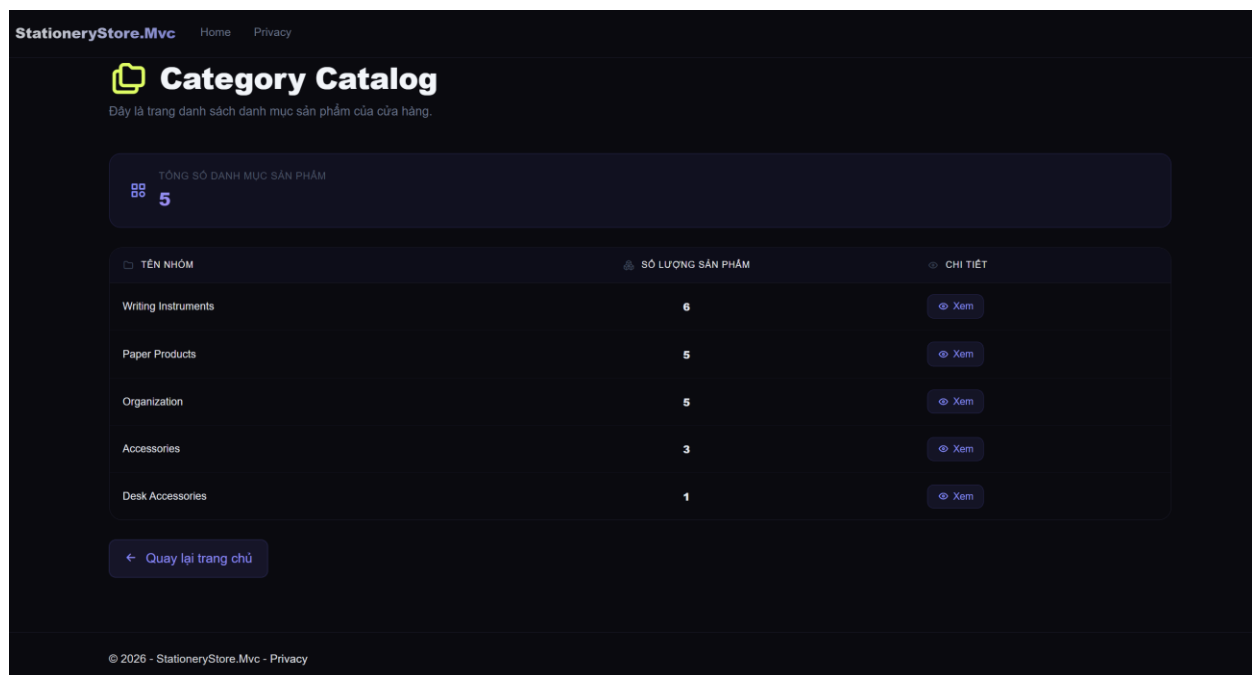
Trang /Stationeries/Detail/999:

```
localhost:5147/Stationeries/Detail/999

Not found stationery with id: 999
```

Ngoài ra: em còn làm thêm Category catalog dùng để lọc ra những văn phòng phẩm nào thuộc danh mục nào, tiện cho việc xem thông tin văn phòng phẩm nếu có quá nhiều sản phẩm.

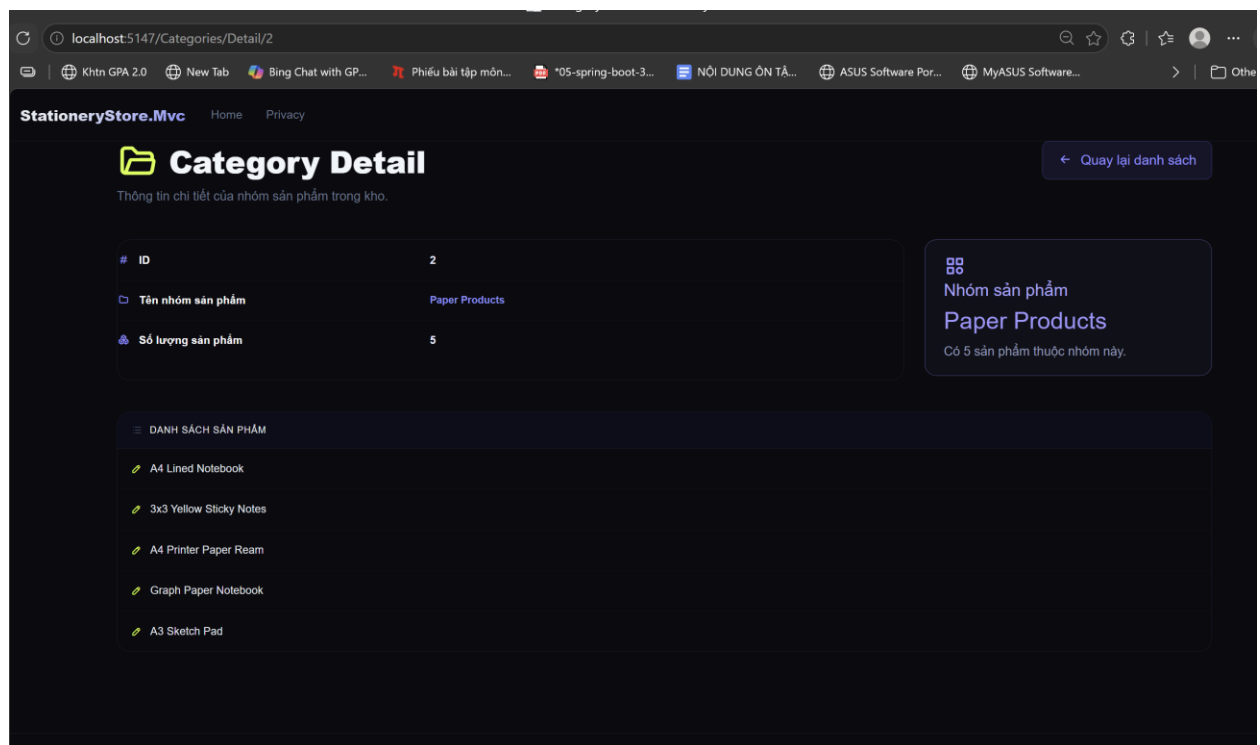
Trang /Categories:



Trang /Categories/Detail/id:

Ví dụ: /Categories/Detail/2:

Với mỗi category chi tiết, chúng ta có thể xem thông tin chi tiết của danh mục đó (id, tên, số lượng sản phẩm, cũng như tên của các sản phẩm)



Trang /Categories/CategoryJson:

```
localhost:5147/Categories/CategoryJson

Pretty-print ☒

[
  {
    "id": 1,
    "name": "Writing Instruments",
    "productCount": 6
  },
  {
    "id": 2,
    "name": "Paper Products",
    "productCount": 5
  },
  {
    "id": 3,
    "name": "Organization",
    "productCount": 5
  },
  {
    "id": 4,
    "name": "Accessories",
    "productCount": 3
  },
  {
    "id": 5,
    "name": "Desk Accessories",
    "productCount": 1
  }
]
```

Trang /Categories/Detail/999:

```
localhost:5147/Categories/Detail/999

Not found category with id: 999
```

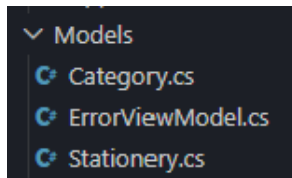
Đặc biệt, ở bài lab này em cũng đã kết nối dữ liệu với database (postgresql) để lấy dữ liệu văn phòng phẩm và danh mục:

Các bước:

+ **Bước 1:** tạo connect string trong appsettings.json để cấu hình port của database, tên database, username, và password.

```
"ConnectionStrings": {
  "DefaultConnection": "Host=localhost;Port=5432;Database=stationeries_store;Username=root;Password=root"
}
```

+ Bước 2: Tạo model Category và Stationery



+ Bước 3: Sau đó ta cần tải các package cần thiết để sử dụng database:

dotnet add package Microsoft.EntityFrameworkCore

dotnet add package Microsoft.EntityFrameworkCore.Design

dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL (ở đây, ta dùng postgresql nên tải package này).

+ Bước 4: Tạo database context

```
using StationeryStore.Mvc.Models;
using Microsoft.EntityFrameworkCore;

namespace StationeryStore.AppDbContext
{
    9 references
    public class ApplicationDbContext : DbContext
    {
        0 references
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options)
        {
        }
        //stationeries table
        7 references
        public DbSet<Stationery> Stationeries {get; set;} = null!;
        //categories table
        2 references
        public DbSet<Category> Categories {get; set;} = null!;
    }
}
```

+ **Bước 5 (optional):** Ta nên chạy lệnh dotnet user-secrets init bởi vì connect string là dữ liệu nhạy cảm nên ta cần bảo mật nó khi chúng ta deploy lên môi trường production. Nếu môi trường development thì không cần cũng được.

dotnet user-secrets set "ConnectionStrings:DefaultConnection" "YourConnectionStringHere"

+ Bước 6: cấu hình dbcontext trong program.cs

```
using StationeryStore.AppDbContext;
using Microsoft.EntityFrameworkCore;
using StationeryStore.Mvc.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

//replace singleton with scoped because of ef core
builder.Services.AddScoped<CategoryService>();
builder.Services.AddScoped<StationeryService>();

var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseNpgsql(connectionString));

var app = builder.Build();
```

+ Bước 7: Khởi tạo database tự động:

dotnet ef migrations add InitialCreate (Này giúp Entity Framework file migration để đồng bộ model với database mà không cần phải code sql thủ công)

dotnet ef database update (thực thi vào database)

Đây là dữ liệu được insert vào Categories table và Stationeries table:

INSERT INTO "Categories"

("Id", "Name")

VALUES

(1, 'Writing Instruments'),

(2, 'Paper Products'),

(3, 'Organization'),

(4, 'Accessories'),

(5, 'Desk Accessories');

INSERT INTO "Stationeries"

("Id", "Sku", "Name", "CategoryId", "Supplier", "Price", "Quantity", "MinStock",
"LastUpdatedAt")

VALUES

(1, 'PEN-BLU-001', 'Blue Ballpoint Pen', 1, 'OfficeSupplies Co.', 1500, 150, 20, '2026-05-18T10:28:11Z'),

(2, 'NBK-A4-LND', 'A4 Lined Notebook', 2, 'PaperWorks Inc.', 45000, 45, 15, '2026-05-18T10:28:11Z'),

(3, 'BND-BLK-1IN', '1-Inch Black Binder', 3, 'OfficeSupplies Co.', 35000, 4, 10, '2026-05-18T10:28:11Z'),

(4, 'ERS-PVD-001', 'Pink PVC Eraser', 4, 'EraserTech', 7000, 0, 50, '2026-05-18T10:28:11Z'),

(5, 'HGL-YLW-001', 'Yellow Highlighter', 1, 'OfficeSupplies Co.', 12000, 80, 10, '2026-05-18T10:28:11Z'),

(6, 'STP-STD-001', 'Standard Desk Stapler', 5, 'OfficeSupplies Co.', 85000, 25, 5, '2026-05-18T10:28:11Z'),

(7, 'PEN-BLK-002', 'Black Ballpoint Pen', 1, 'OfficeSupplies Co.', 1500, 200, 20, '2026-05-19T09:00:00Z'),

(8, 'PEN-RED-003', 'Red Ballpoint Pen', 1, 'OfficeSupplies Co.', 1500, 120, 20, '2026-05-19T09:00:00Z'),

(9, 'PNC-MCH-001', '0.5mm Mechanical Pencil', 1, 'OfficeSupplies Co.', 18000, 60, 15, '2026-05-19T09:00:00Z'),

(10, 'MRK-PRM-BLK', 'Permanent Marker Black', 1, 'OfficeSupplies Co.', 15000, 40, 10, '2026-05-19T09:00:00Z'),

(11, 'NOT-STK-3X3', '3x3 Yellow Sticky Notes', 2, 'PaperWorks Inc.', 12000, 100, 30, '2026-05-19T09:00:00Z'),

(12, 'PPR-A4-500', 'A4 Printer Paper Ream', 2, 'PaperWorks Inc.', 75000, 500, 50, '2026-05-19T09:00:00Z'),

(13, 'NBK-GRH-001', 'Graph Paper Notebook', 2, 'PaperWorks Inc.', 50000, 30, 10, '2026-05-19T09:00:00Z'),

(14, 'PPR-SKT-A3', 'A3 Sketch Pad', 2, 'PaperWorks Inc.', 85000, 20, 5, '2026-05-19T09:00:00Z'),

(15, 'FLD-MAN-001', 'Manila File Folders (10 Pk)', 3, 'OfficeSupplies Co.', 25000, 80, 20, '2026-05-19T09:00:00Z'),

(16, 'CLP-PPR-100', 'Paper Clips (100 Box)', 3, 'OfficeSupplies Co.', 8000, 150, 30, '2026-05-19T09:00:00Z'),

(17, 'BOX-STR-PLX', 'Plastic Storage Box', 3, 'OfficeSupplies Co.', 120000, 15, 5, '2026-05-19T09:00:00Z'),

(18, 'FLD-EXP-PST', 'Expanding File Folder', 3, 'OfficeSupplies Co.', 45000, 35, 10, '2026-05-19T09:00:00Z'),

(19, 'TPE-CLR-001', 'Clear Tape Roll', 4, 'EraserTech', 10000, 90, 20, '2026-05-19T09:00:00Z'),

(20, 'SCS-STD-08', '8-Inch Scissors', 4, 'EraserTech', 35000, 45, 10, '2026-05-19T09:00:00Z');

Câu 2:

1. Vì sao không nên đưa toàn bộ dữ liệu và logic xử lý trực tiếp vào Controller? Trong bài của em, Service đang giúp tách trách nhiệm như thế nào?

+ Thứ nhất, bản chất Controller chỉ làm nhiệm vụ nhận request và trả về response.

+ Thứ hai, làm cho code trở nên khó mở rộng và bảo trì, vì không có cấu trúc rõ ràng. Nếu dồn hết tất cả vào controller thì controller phải làm quá nhiều việc.

Ví dụ: sau này, ta update 1 tính năng mới là gửi email tự động vào buổi sáng lúc 9h để báo số lượng tồn kho hiện tại của số lượng văn phòng phẩm cho ông chủ. Và tất nhiên email tự động là 1 tác vụ chạy ngầm (background job). Và đương nhiên email tự động này sẽ không thể gọi vào StationeryController như những request do người dùng bấm nút hay gõ Url được. Vậy thì chúng ta sẽ phải copy paste đoạn code tính số lượng tồn kho ở StationeryController và vào class để xử lý việc gửi email. Suy ra code sẽ bị lặp lại rất nhiều.

Nếu như sau này ta đổi logic từ $Quantity \leq MinStock$ thành $Quantity \leq Minstock + 5$ thì chúng ta sẽ khó có thể nhớ hết được logic này nằm ở đâu trong chương trình của chúng ta.

Và service sẽ giúp chúng ta như thế nào: class xử lý email -> gọi `_stationeryService.GetStats()` -> service sẽ nhét số liệu vào email và gửi đi.

Nếu sau này chúng ta có thay đổi logic thì cũng chỉ cần sửa trong service là xong.

Trong bài của em, CategoryService và StationeryService đang giúp tách trách nhiệm:

CategoryController và StationeryController giúp nhận request từ users, sau khi nhận request, controller sẽ tiến hành gọi service để xử lý request đó.

Ví dụ: Người dùng nhấn xem thống kê tồn kho, Controller sẽ nhận được request `/Stationeries/Stats`, sau đó nó sẽ gọi hàm `GetStats()` từ service để xử lý request này và response

lại cho người dùng. Hàm GetStats() trong StationeryService sẽ giúp chúng ta tính các thống kê tồn kho từ các dữ liệu được gọi từ database và trả lại kết quả khi được gọi từ controller. Đây chính là trách nhiệm của service.

2. Vì sao không nên truyền trực tiếp Model gốc sang View trong mọi trường hợp?

ViewModel trong bài của em đang giải quyết vấn đề gì?

Khi truyền trực tiếp Model gốc sang View sẽ gây ra những vấn đề sau:

+ Khó cải tiến và mở rộng: nếu chúng ta phát triển tính năng thêm mới văn phòng phẩm, thì nếu như ta cần thêm một danh sách thả xuống (dropdown) để chọn danh mục (category), nhưng chúng ta lại dùng model gốc và không hề có danh sách đó, vậy thì chúng ta sẽ không thể làm được.

+ Tính bảo mật: ví dụ như chúng ta có tính năng cập nhật văn phòng phẩm, nhưng nếu người dùng cập nhật những dữ liệu mang tính bảo mật của văn phòng phẩm, và vì chúng ta đang dùng model gốc nên sẽ cập nhật trực tiếp vào model này luôn.

Trong bài của em, ViewModel đã giải quyết được vấn đề:

+ Tính toán được tổng giá trị tồn kho (Inventory Value), thuộc tính này không có trong model gốc.

+ Thuộc tính kiểm tra tình trạng tồn kho (còn tồn, hết hàng, cần nhập thêm). Thuộc tính này cũng không có trong model gốc.

3. Trong trang danh sách, em đã xử lý trạng thái nghiệp vụ như “Còn hàng / Hết hàng”, “Còn chỗ / Đã đầy”, hoặc trạng thái tương tự ở đâu? Vì sao không nên viết quá nhiều logic này trong View?

Em đã xử lý trạng thái nghiệp vụ “Còn hàng / Hết hàng”, “Còn chỗ / Đã đầy” ở ViewModel. Nhưng nếu logic nghiệp vụ này cần đến database thì em nghĩ nên xử lý ở tầng service sẽ tốt hơn.

```
using System;
public string StockStatus
{
    get
    {
        if (Quantity <= 0)
        {
            return "Out of stock";
        }
        if (Quantity <= MinStock)
        {
            return "Need to add more stock";
        }
        return "In stock";
    }
}

using System;
public string StockStatusClass
{
    get
    {
        if (Quantity <= 0)
        {
            return "badge badge-danger";
        }
        if (Quantity <= MinStock)
        {
            return "badge badge-warning";
        }
        return "badge badge-success";
    }
}
```

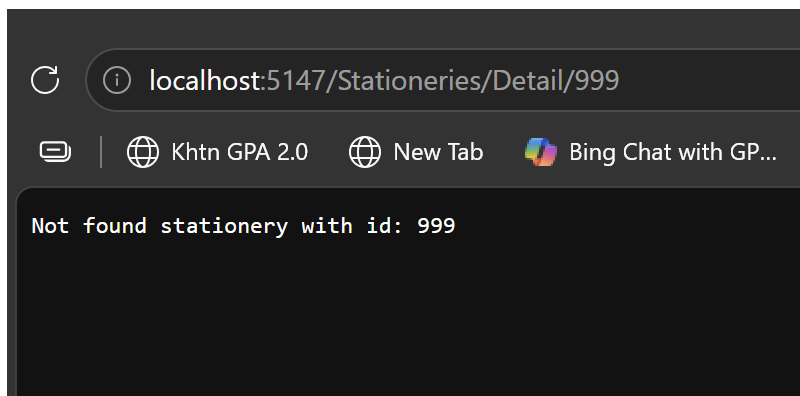
Nếu viết quá nhiều logic trong view sẽ dẫn tới:

+ Code khó bảo trì, cải tiến, mở rộng vì view đã chứa nhiều thẻ <div>, <table> rồi còn thêm các style css, nó đã khá rối rắm và nhiều, nếu chúng ta còn xử lý quá nhiều logic trong đây sẽ dẫn tới việc khi muốn nâng cấp sẽ khó vì phải đi lục và đọc để tìm ra chỗ để update logic.

+ Vai trò chính của view chủ yếu sẽ là nhiệm vụ trình bày, nên em nghĩ logic nên xử lý ở ViewModel hoặc Service.

4. Nếu người dùng truy cập một Id không tồn tại, ví dụ /Courses/Detail/999, hệ thống của em xử lý thế nào? Vì sao nên dùng NotFound() ?

Nếu người dùng truy cập một Id không tồn tại, hệ thống sẽ tự động báo lỗi.



Vì notfound sẽ cho chúng ta biết lỗi cụ thể ở đây là gì, vì nếu không dùng thì web của chúng ta sẽ hiển thị ra lỗi rất là dài, và nếu như những người không có chuyên môn thì sẽ không thể biết đó là lỗi gì. Nếu dùng notfound() thì ai cũng có thể hiểu được là id này không tồn tại.

5. Nếu dữ liệu ngày càng nhiều, việc lưu dữ liệu bằng List hard-code trong Service sẽ gặp hạn chế gì?

Nếu dữ liệu ngày càng nhiều (1000 loại giấy khác nhau, 100 mẫu bút khác nhau,...), việc lưu bằng List hoặc dữ liệu hard-code trong service sẽ gặp những hạn chế sau:

- + Tốn bộ nhớ Ram, vì dữ liệu hardcode hay List sẽ được lưu trên Ram, vì vậy nên khi quá nhiều dữ liệu thì sẽ tốn rất nhiều bộ nhớ Ram, điều này làm ảnh hưởng đến hiệu năng của hệ thống và trải nghiệm của người dùng khi dùng chương trình.
- + Dữ liệu sẽ không được lưu trữ khi khởi động chương trình lại, ví dụ cập nhật giá tiền cho cây bút mực thì khi khởi động lại chương trình thì giá tiền vẫn là giá tiền của dữ liệu hardcode chứ không phải dữ liệu mới cập nhật.

Với bài lab này em đã dùng giải pháp là kết nối với database để xử lý dữ liệu. Giải pháp này sẽ tốt cho chương trình nếu chúng ta muốn cải tiến và mở rộng ứng dụng sau này.

6. Nếu muốn phát triển bài này thành hệ thống thật ở các tuần sau, theo em nên ưu tiên mở rộng phần nào trước: CRUD, tìm kiếm, phân trang, validation, authentication/authorization hay kết nối database? Hãy giải thích lý do.

Vì ở lab01, em đã ưu tiên kết nối database, và bây giờ em đã kết nối thành công bài lab02 với database, nên hiện tại em sẽ ưu tiên mở rộng phần CRUD, vì với database thì việc crud sẽ dễ dàng và không phải lo dữ liệu sẽ mới cập nhật sẽ bị mất (nếu dùng hardcode).

7. Theo em, trong bài toán em chọn, thống kê nào là quan trọng nhất để người dùng ra quyết định? Vì sao?

Dựa vào 5 con số em đã thống kê (tổng số văn phòng phẩm, tổng số lượng tồn kho, tổng giá trị tồn kho, số lượng văn phòng phẩm hết hàng, số lượng văn phòng phẩm cần nhập hàng thêm). Thì số lượng văn phòng phẩm cần nhập hàng thêm sẽ là quan trọng nhất để người dùng đưa ra quyết định vì nếu chúng ta chủ động nhập hàng sớm thì tình trạng hết hàng sẽ không còn và được khắc phục. Sẽ không dẫn đến làm mất lòng tin ở khách hàng.

8. Nếu View bị lỗi do sai kiểu dữ liệu, ví dụ Controller gửi List<CourseViewModel> nhưng View khai báo @model CourseViewModel, em sẽ kiểm tra và sửa theo thứ tự nào?

+ Đầu tiên, em kiểm tra xem trang này đang cần hiển thị là 1 món văn phòng phẩm, hay nhiều món văn phòng phẩm.

Giả sử nếu trang đang lỗi là cần hiển thị chi tiết danh sách văn phòng phẩm.

+ Thứ hai, kiểm tra controller và nhìn vào tham số được truyền vào khi return View(tham số) là kiểu gì. Nếu controller trả về 1 list văn phòng phẩm thì đúng.

+ Thứ ba, nếu view sử dụng @model StationeryDetailViewModel thì phải sửa lại thành @model StationeryListItemViewModel.